

PL/SQL Basics Overview

PL/SQL: Introduction

Introduction to PL SQL.

A few points about PL SQL:

- PL/SQL lets you write code once and deploy it in the database nearest the data.

PL/SQL can simplify application development, optimize execution, and improve resource utilization in the database.

- The language is a case-insensitive programming language, like SQL.

- History: PL/SQL was developed by modeling concepts of structured programming, static data typing, modularity, exception management, and parallel (concurrent) processing found in the Ada programming language. The Ada programming language, developed for the United States Department of Defense, was designed to support military real-time and safety-critical embedded systems, such as those in airplanes and missiles. The Ada programming language borrowed significant syntax from the Pascal programming language, including the assignment and comparison operators and the single-quote delimiters.

4.0.1 Block

PL/SQL is a blocked programming language. Program units can be named or unnamed

blocks. Unnamed blocks are known as anonymous blocks and are labeled so throughout

the book. The PL/SQL coding style differs from that of the C, C++, and Java programming

languages. For example, curly braces do not delimit blocks in PL/SQL.

Anonymous-block programs are effective in some situations. You typically use anonymous blocks when building scripts to seed data or perform one-time processing activities.

They are also effective when you want to nest activity in

another PL/SQL blocks execution section. The basic anonymous-block structure must contain an execution section. You can also put optional declaration and exception sections in anonymous blocks. The following illustrates an anonymous-block prototype:

29

Dr. Raihan Kamal

CHAPTER 4. PL/SQL: INTRODUCTION

[DECLARE]

declaration_statements

BEGIN

execution_statements

[EXCEPTION]

exception_handling_statements

END;

/

Lets write out first block that will do nothing:

BEGIN

NULL;

END;

/

Hello World:

SET SERVEROUTPUT ON SIZE 1000000

BEGIN

dbms_output.put_line('Hello World.');

END;

/

scanf:--scanf-

DECLARE

my_var VARCHAR2(30);

BEGIN

my_var := '&i';

dbms_output.put_line('Hello ' || my_var);

END;

4.0.2 Variables, Assignments, and Operators

Delimiters: Lexical delimiters are symbols or symbol sets. For complete set see table 3-1. Here are some important symbols frequently used in pl sql programming.

30

Dr. Raihan Kamal

CHAPTER 4. PL/SQL: INTRODUCTION

- := Assignment

The assignment operator is a colon immediately followed by an equal symbol. It is the only assignment operator in the language. You assign a right operand to a left operand, like `a := b + c;`

- . Association

The component selector is a period, and it glues references together, for example, a schema and a table, a package and a function, or an object and a member method.

Component selectors are also used to link cursors and cursor attributes (columns).

The following are some prototype examples:

`schema_name.table_name`

`package_name.function_name`

`object_name.member_method_name`

`cursor_name.cursor_attribute`

`object_name.nested_object_name.object_attribute`

These are referenced in subsequent chapters throughout this book

Understand these. dont say anything



give each exampls one

@ Association. The remote access indicator lets you access a remote database through database links.

- = Comparison.
- <> != Comparison Not equal to.
- - Delimiter In-line comment.
- /* Delimiter */ Multi-line comment.
- " Delimiter

Here's **one example for each** of the symbols/concepts you mentioned:

✓ @ Association (Remote Access via DB Link)

sql

```
SELECT * FROM employees@hr_db_link;
```

Accesses the `employees` table from a remote database via the `hr_db_link`.

✓ = Comparison Operator

plsql

```
IF salary = 5000 THEN  
  dbms_output.put_line('Exact match');  
END IF;
```

✓ <> or != Not Equal To

plsql

```
IF department_id <> 10 THEN  
  dbms_output.put_line('Different department');  
END IF;
```

You can also use `!=` in Oracle, though `<>` is the standard.

✓ -- In-line Comment

pl

```
salary := 5000; -- Set base salary
```

`/* ... */`

Multi-line Comment

plsql

```
/*  
  This block sets the bonus  
  for high performing employees  
*/  
bonus := 1000;
```

`"`

Quoted Identifier

sql

```
CREATE TABLE "Employee Data" (  
  "Emp ID" NUMBER,  
  "Name" VARCHAR2(100)  
);
```

Used to allow spaces or case-sensitive identifiers.

Let me know if you want a runnable block using these!

give me difference between named block and unnamed block

Here is a **clear comparison between Named and Unnamed Blocks** in PL/SQL:



Unnamed Block (Anonymous Block)

Feature	Description
Definition	A block without a name .
Usage	One-time use, quick operations like data seeding, testing, or scripting.
Storage	Not stored in the database.
Calling	Cannot be called by name.
Structure	Must have <code>BEGIN...END</code> , optionally <code>DECLARE</code> and <code>EXCEPTION</code> .
Compilation	Compiled and executed immediately.

✓ Example:

```

psql

BEGIN
  dbms_output.put_line('Hello from anonymous block!');
END;
/

```

◆ Named Block

Feature	Description
Definition	A block with a name , like a Procedure , Function , Package , or Trigger .
Usage	Reusable, modular programming; called from applications or other PL/SQL blocks.
Storage	Stored in the database.
Calling	Can be called by name (e.g., <code>EXEC procedure_name;</code>).
Structure	Starts with a name and often includes parameters.
Compilation	Compiled and stored before execution.

✓ Example (Procedure):

```
plsql
```

```
CREATE OR REPLACE PROCEDURE greet_user IS
BEGIN
  dbms_output.put_line('Hello from named block!');
END;
/
```

Summary Table:

Feature	Named Block	Unnamed Block
Has a name?	✔ Yes	✗ No
Stored in DB?	✔ Yes	✗ No
Reusable?	✔ Yes	✗ No
Invoked by name?	✔ Yes	✗ No
Use case	Procedures, packages, etc.	One-time scripts
Compiled on	Creation	Execution

Let me know if you want a visual diagram or real-life analogy!

give printing and scanning example

Here are **PL/SQL examples** showing both **printing (output)** and **scanning (input using substitution variable)**:

✔ Printing Example (using `DBMS_OUTPUT.PUT_LINE`)

```
plsql
```

```
-- Enable output
SET SERVEROUTPUT ON;

BEGIN
  dbms_output.put_line('Hello, PL/SQL!');
END;
/
```

✓ Scanning Example (input using substitution variable &)

```
pl

-- Enable output
SET SERVEROUTPUT ON;

DECLARE
  user_name VARCHAR2(30);
BEGIN
  -- Scans input value from user at runtime
  user_name := '&name';

  -- Prints the input back
  dbms_output.put_line('Hello, ' || user_name || '!');
END;
/
```

🔍 What happens:

- When this block runs, SQL*Plus or tools like SQL Developer will **prompt you for input**:

```
rust
```

Enter value **for name**:

- If you type **Arham**, the output will be:

```
Hello, Arham!
```


Let me know if you want an example using `ACCEPT` or `&1` for positional inputs too.

give me difference between varchar and varchar2

Here's a **clear and concise difference** between `VARCHAR` and `VARCHAR2` in Oracle:

 VARCHAR vs VARCHAR2 in Oracle

Feature	VARCHAR	VARCHAR2 (Recommended)
Defined In	SQL standard (but reserved)	Oracle-specific extension
Availability	Exists but should not be used	Fully supported and widely used
Behavior	Reserved for future use	Stores variable-length strings
NULL vs Empty	Might treat them differently in the future	Currently treats NULL and empty string as same
Length Limit	Up to 4000 bytes (same as VARCHAR2)	Up to 4000 bytes
Best Practice	 Avoid using it	 Use this always

 Why VARCHAR is not recommended:

Oracle reserves `VARCHAR` for future use in a way that may treat **empty strings and NULLs differently**, which can break existing logic. Currently, `VARCHAR` **behaves exactly like** `VARCHAR2` , but Oracle may change that.

 Always Use:

sql

VARCHAR2(size)

Example:

sql

```
CREATE TABLE test_table (  
  name VARCHAR2(50)  
);
```

Avoid this:

sql

```
CREATE TABLE test_table (  
  name VARCHAR(50) -- ❌ avoid  
);
```

Let me know if you want a test case to demonstrate how they behave in practice.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  c CHAR(2000) := 'hello';
```

```
  v VARCHAR2(32767) := 'hello';
```

```
BEGIN
```

```
  dbms_output.put_line('The length of c is: ' || LENGTH(c));
```

```
  dbms_output.put_line('The length of v is: ' || LENGTH(v));
```

```
END;
```

```
/
```

Great! Let me explain what this PL/SQL block does, and what output you can expect:

Code Breakdown:

sql

```
SET SERVEROUTPUT ON;

DECLARE
  c CHAR(2000) := 'hello';      -- Fixed-length 2000 characters
  v VARCHAR2(32767) := 'hello'; -- Variable-length string
BEGIN
  dbms_output.put_line('The length of c is: ' || LENGTH(c));
  dbms_output.put_line('The length of v is: ' || LENGTH(v));
END;
/
```

Expected Output:

r

```
The length of c is: 2000
The length of v is: 5
```

Why?

- `CHAR(2000)` : This always stores exactly **2000 characters**, right-padded with spaces. So `'hello'` is stored as `'hello ... (1995 spaces)'`.
→ `LENGTH(c)` returns **2000**.
- `VARCHAR2(32767)` : Stores only the actual characters. `'hello'` has 5 characters.
→ `LENGTH(v)` returns **5**.

Tip:

- Use `CHAR` only when you **always need fixed-length** (e.g., flags, codes like 'Y/N').
- Use `VARCHAR2` for **almost everything else**.

Let me know if you'd like to test `TRIM(c)` or `LENGTH(TRIM(c))` to see how spaces are handled!

`NUMBER`. It can store numbers in the range of 1.0E-130 (1 times 10 raised to the negative 130th power) to 1.0E126 (1 times 10 raised to the 126th power).

The following is the prototype for declaring a fixed-point `NUMBER` datatype:

`NUMBER[(precision, [scale])] [NOT NULL]`

Explain with example

1 2
3 4

`NUMBER` in Oracle PL/SQL — Explained with Examples

The `NUMBER` data type is **very flexible** and can be used to store **integers, decimals, or very large/small values**.

General Syntax:

sql

`NUMBER[(precision, scale)] [NOT NULL]`

Term	Meaning
precision	Total number of digits (both sides of the decimal point).
scale	Number of digits after the decimal point (fractional part).

Behavior Based on Precision & Scale:

Declaration	Meaning	Example Values
<code>NUMBER</code>	No limits; stores any valid number	<code>123</code> , <code>1.5</code> , <code>999999</code>
<code>NUMBER(5)</code>	Max 5 digits (e.g., <code>99999</code>)	✓ <code>12345</code> ✗ <code>123456</code>
<code>NUMBER(5, 2)</code>	Total 5 digits, 2 after decimal point	✓ <code>123.45</code> , ✗ <code>1234.56</code>
<code>NUMBER(5, 0)</code>	Integer up to 5 digits	✓ <code>12345</code> ✗ <code>123.45</code>
<code>NUMBER(5, -2)</code>	Rounds to hundreds	✓ <code>12300</code> , <code>45000</code>

✓ Examples:

Example 1: Basic Number

```
pl

DECLARE
  a NUMBER := 1234567890;
BEGIN
  dbms_output.put_line(a);
END;
/
```

Example 2: Fixed-point Number

```
plsql

DECLARE
  salary NUMBER(7,2) := 12345.67;
BEGIN
  dbms_output.put_line('Salary: ' || salary);
END;
/
```

- `7` total digits

cost NUMBER(4,2) := 123.45; -- **✗** Error: total digits = 5

BEGIN

```
dbms_output.put_line('Cost: ' || cost);
```

END;

Oracle `NUMBER` can store values from:

- [illegible]

- to 1.0E+126 = 100000000000... (very large)

NUMBER[(precision, [scale])]

explain this part

explain this part

1
2
3
4

NUMBER[(precision, [scale])] — Fully Explained

In Oracle PL/SQL, this is the **optional format** used to define how precise a number should be.

Components:

Part	Meaning
precision	Total number of digits allowed , including both left and right of the decimal point.
scale	Number of digits allowed after the decimal point (i.e., fractional digits).

✓ Both are optional. If not given, Oracle allows **very large or small numbers** by default.

Think of it as:

SCSS

NUMBER(total_digits, digits_after_decimal)

Examples:

1. NUMBER(5, 2)

- Total **5 digits**
- **2 digits after** decimal
- So: up to **3 digits before** decimal and **2 after**

✓ Valid: 123.45

✗ Invalid: 1234.56 (because that's 6 digits total)

2. `NUMBER(5)`

- Only **precision** is given
- Treated as `NUMBER(5, 0)`
- Allows **integer** up to 5 digits

✓ Valid: `99999`

✗ Invalid: `100000` , `123.4`

3. `NUMBER(*, -2)`

- Rare, but allowed: **negative scale**
- Rounds number to nearest **hundred, thousand, etc.**

```
sql
```

```
NUMBER(6, -2)
```

Means:

- 6 digits total
- Rounded to the **nearest hundred**

✓ Valid: `123400` , `999900`

✗ Invalid: `123456`

⚠ Note:

- If **precision and scale are omitted**, like just `NUMBER` , Oracle allows up to:
 - **38 digits precision**
 - Very small or large numbers (scientific range)
-

Summary Table:

Declaration	Meaning	Example Valid Value
<code>NUMBER</code>	Any number, flexible	<code>123456789</code> , <code>0.001</code>
<code>NUMBER(6)</code>	Integer with max 6 digits	<code>123456</code>
<code>NUMBER(6, 2)</code>	Max 6 digits, 2 after decimal	<code>1234.56</code>
<code>NUMBER(6, -2)</code>	Rounded to nearest 100	<code>120000</code>

Let me know if you'd like to test these inside a table or block!

write me the structure of in else elseif statement and then give an example

✅ PL/SQL `IF...ELSIF...ELSE` Statement — Structure + Example

📌 Structure:

plsql

```

IF condition1 THEN
    -- statements if condition1 is TRUE
ELSIF condition2 THEN
    -- statements if condition2 is TRUE
ELSIF condition3 THEN
    -- statements if condition3 is TRUE
ELSE
    -- statements if none of the above are TRUE
END IF;
```

- `ELSIF` (not `ELSE IF`) is the correct keyword in PL/SQL.
- You can have multiple `ELSIF` blocks.
- The `ELSE` block is optional.

✓ Example:

plsql

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  grade CHAR(1) := 'B';
```

```
BEGIN
```

```
  IF grade = 'A' THEN
```

```
    dbms_output.put_line('Excellent');
```

```
  ELSIF grade = 'B' THEN
```

```
    dbms_output.put_line('Good Job');
```

```
  ELSIF grade = 'C' THEN
```

```
    dbms_output.put_line('Needs Improvement');
```

```
  ELSE
```

```
    dbms_output.put_line('Invalid Grade');
```

```
  END IF;
```

```
END;
```

```
/
```



Output:

nginx

Good Job

Let me know if you want a numeric version or nested `IF` example too!

write me the structure of CASE statement and then give an example

✓ PL/SQL CASE Statement — Structure + Example

Structure of Simple CASE Statement:

plsql

```
CASE expression
WHEN value1 THEN
  -- statements for value1
WHEN value2 THEN
  -- statements for value2
...
ELSE
  -- statements if no match
END CASE;
```

`expression` is evaluated and matched against each `value`. If a match is found, the corresponding block is executed.

Structure of Searched CASE Statement:

pl

```
CASE
WHEN condition1 THEN
  -- statements
WHEN condition2 THEN
  -- statements
...
ELSE
  -- statements if no condition matches
END CASE;
```

This is useful when conditions are more complex and not just value matches.

✓ Example of Simple CASE:

plsql

```
SET SERVEROUTPUT ON;

DECLARE
  grade CHAR(1) := 'C';
BEGIN
  CASE grade
    WHEN 'A' THEN dbms_output.put_line('Excellent');
    WHEN 'B' THEN dbms_output.put_line('Good');
    WHEN 'C' THEN dbms_output.put_line('Average');
    ELSE dbms_output.put_line('Invalid grade');
  END CASE;
END;
/
```

✓ Example of Searched CASE:

plsql

```
SET SERVEROUTPUT ON;

DECLARE
  marks NUMBER := 65;
BEGIN
  CASE
    WHEN marks >= 90 THEN dbms_output.put_line('Grade A');
    WHEN marks >= 75 THEN dbms_output.put_line('Grade B');
    WHEN marks >= 60 THEN dbms_output.put_line('Grade C');
    ELSE dbms_output.put_line('Fail');
  END CASE;
```

```
END;  
/
```

Let me know if you want to see how to use `CASE` inside a `SELECT` query as well!

Loop

PL/SQL Loops — Complete Guide with Structure & Examples

PL/SQL provides **three types of loops**:

1. **Basic LOOP**

Structure:

```
plsql  
  
LOOP  
  -- statements  
  EXIT WHEN condition;  
END LOOP;
```

Example:

```
plsql  
  
SET SERVEROUTPUT ON;  
  
DECLARE  
  i NUMBER := 1;  
BEGIN  
  LOOP  
    dbms_output.put_line('i = ' || i);  
    i := i + 1;  
    EXIT WHEN i > 5;
```

```
END LOOP;  
END;  
/
```

2. WHILE LOOP

Structure:

```
plsql  
  
WHILE condition LOOP  
  -- statements  
END LOOP;
```

Example:

```
plsql  
  
SET SERVEROUTPUT ON;  
  
DECLARE  
  i NUMBER := 1;  
BEGIN  
  WHILE i <= 5 LOOP  
    dbms_output.put_line('i = ' || i);  
    i := i + 1;  
  END LOOP;  
END;  
/
```

3. FOR LOOP

Structure:

```
plsql
```

```
FOR counter_variable IN [REVERSE] lower_bound..upper_bound LOOP
  -- statements
END LOOP;
```

✓ Example (Forward):

```
plsql

SET SERVEROUTPUT ON;

BEGIN
  FOR i IN 1..5 LOOP
    dbms_output.put_line('i = ' || i);
  END LOOP;
END;
/
```

✓ Example (Reverse):

```
plsql

SET SERVEROUTPUT ON;

BEGIN
  FOR i IN REVERSE 5..1 LOOP
    dbms_output.put_line('i = ' || i);
  END LOOP;
END;
/
```

⚠ Loop Control Keywords:

- `EXIT` – exits a loop
- `EXIT WHEN <condition>` – conditional exit
- `CONTINUE` – skip current iteration (Oracle 11g+)

Let me know if you'd like nested loops or cursor-based loops next!

